

范思眸项目学习记录-第 21 次汇报

小组成员：范思眸

2026 年 4 月 5 日-2026 年 4 月 11 日

目录

1	DBOT 实验结果	2
1.1	变换一	2
1.2	变化二	4
2	附录	6

1 DBOT 实验结果

- 按照老师要求，我将基于 cv 变换获得上下确界，具体实验操作步骤如下。

针对豹子图，我将定义两个变换。针对噪声图，我也将定义两个变换。基于此，我们总共获得了 4 副图片。

为了获得严格的上下确界限，我们逐个比较矩阵每个位置元素的大小，将两幅图片每个位置的元素的最大值的矩阵称为上界，反之则称为下界。图 1 展示了 4 副图片经过上述 cv 变化后的上下确界图，图 2 展示了图像的平滑变化过程，图 3 为 Distance 距离图。

1.1 变换一

针对豹子图：变换一是对豹子图轻微旋转，变换二是对豹子图轻微亮度调整。针对噪声图：变换一是对噪声图轻微平滑，变化二是对噪声图轻微平移。具体实验结果见图 1 至图 3。

Upper Bound and Lower Bound under CV Transformation

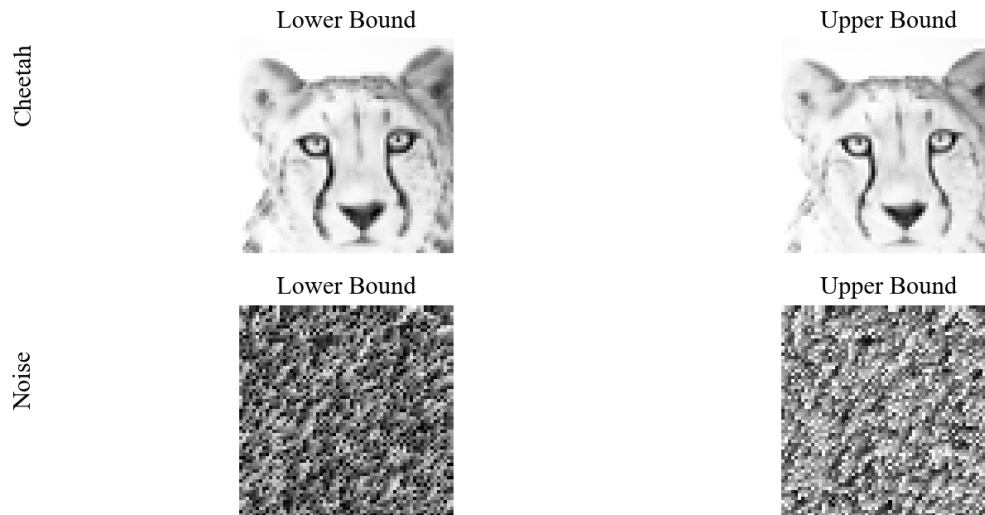


图 1: 上下确界

Smooth Transformation under CV-based Bounds

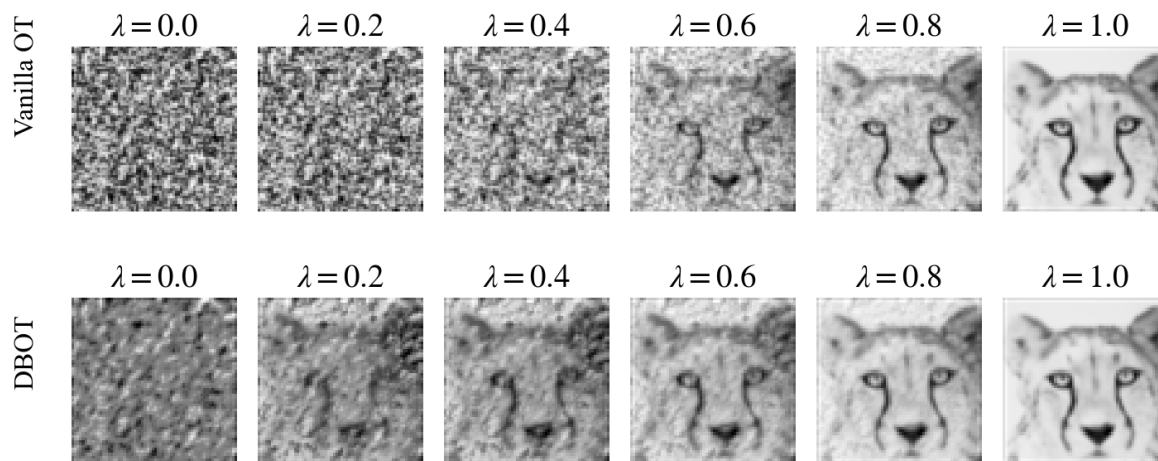


图 2: DBOT 与 Valinna OT 的变换图

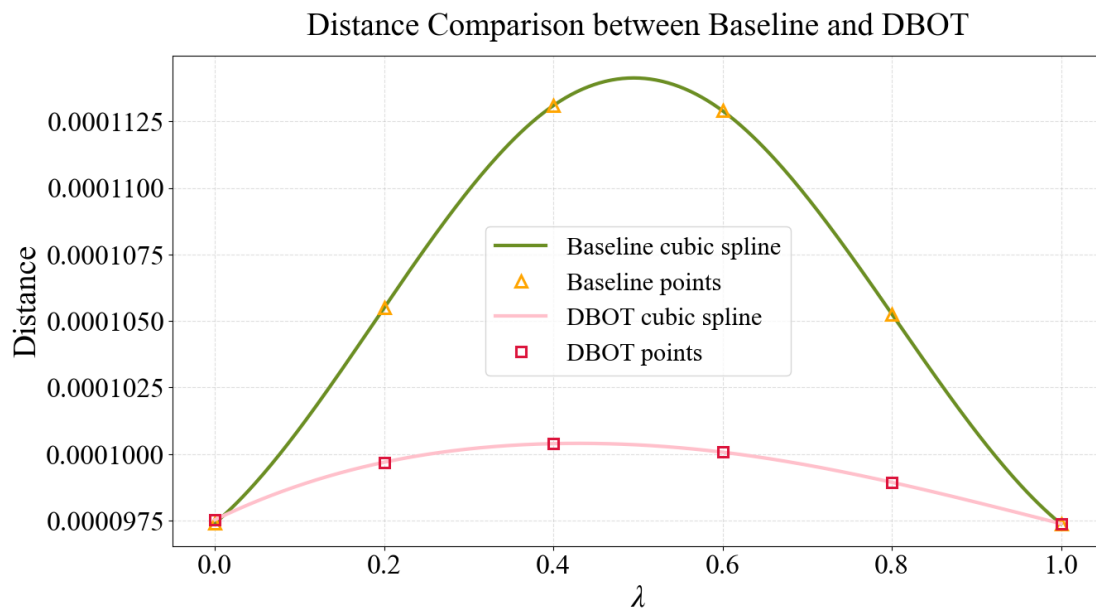


图 3: DBOT 与 Valinna OT 的距离图

1.2 变化二

针对豹子图：变换一是对豹子图轻微平移，变换二是对豹子图轻微高斯模糊。针对噪声图：变换一是对噪声图轻微亮度调整，变换二是对噪声图轻微高斯模糊。具体实验结果见图 4 至图 6。

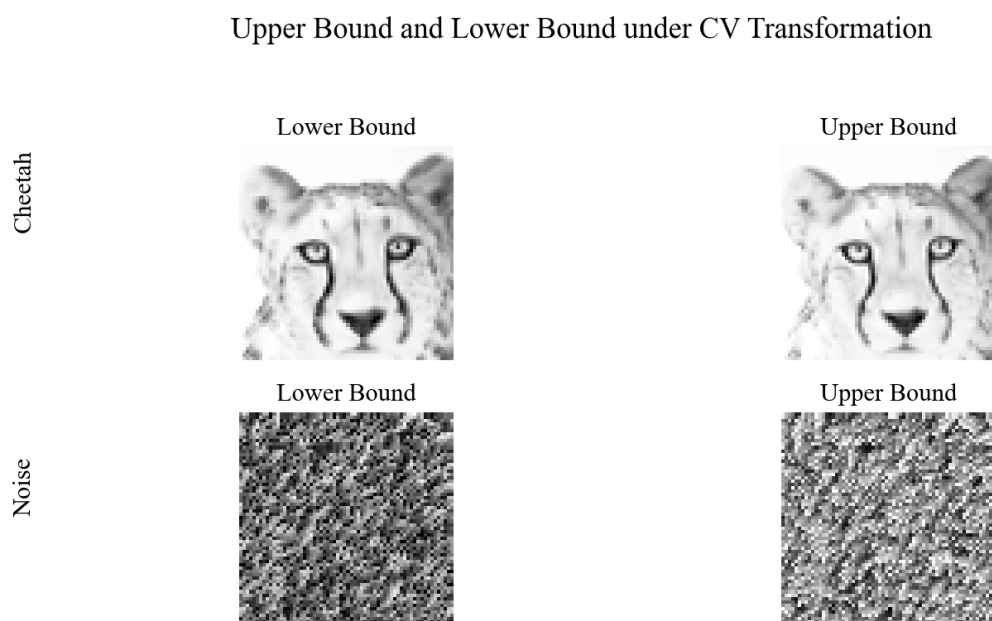


图 4: 上下确界

Smooth Transformation under CV-based Bounds

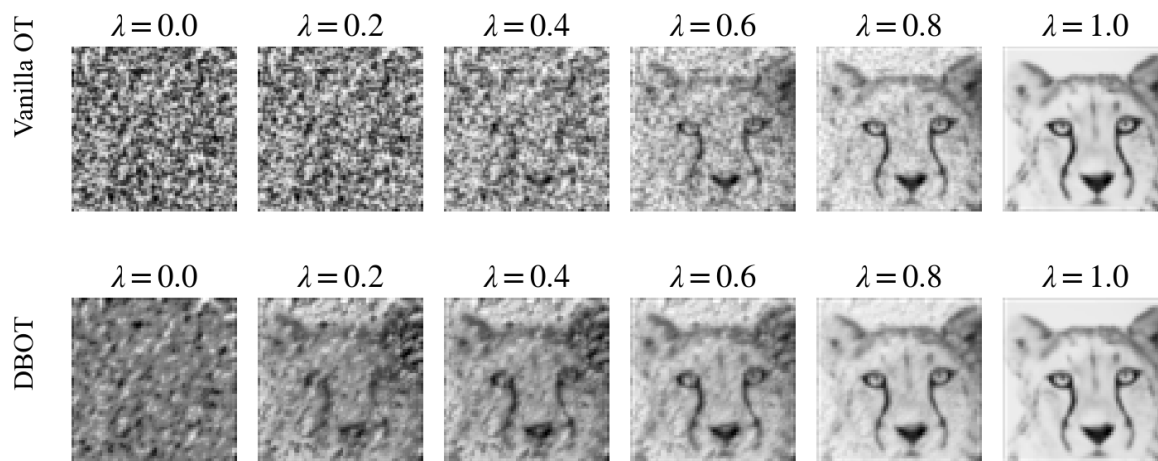


图 5: DBOT 与 Valinna OT 的变换图

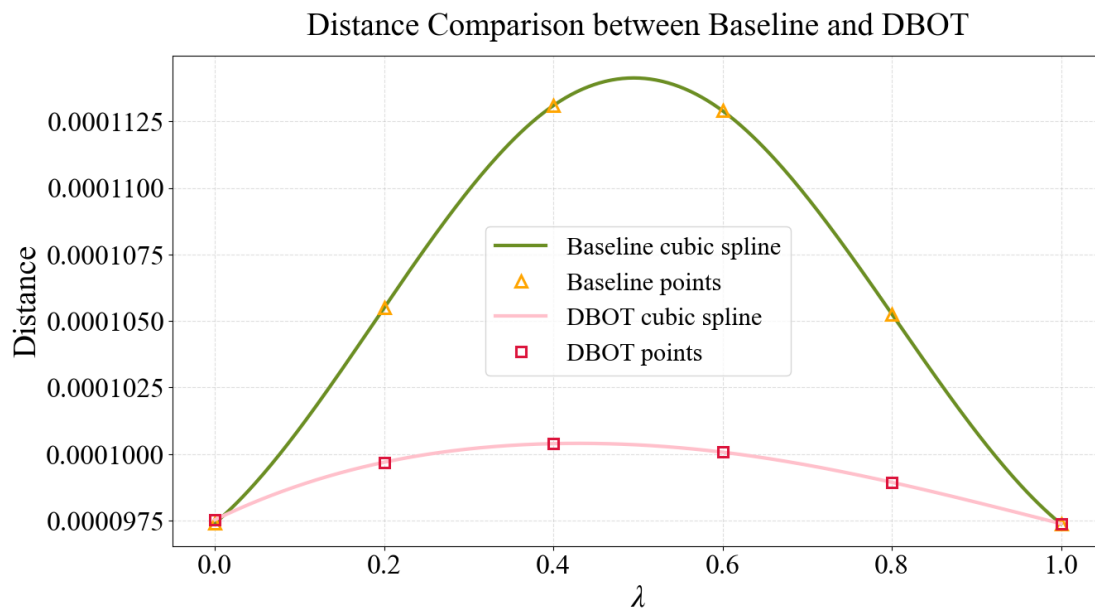


图 6: DBOT 与 Valinna OT 的距离图

2 附录

Experiment 相关代码

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import time
4 from scipy.interpolate import CubicSpline
5 import matplotlib as mpl
6 from PIL import Image
7 from scipy.ndimage import rotate, shift, gaussian_filter
8
9 mpl.rcParams['font.family'] = 'Times New Roman'
10 mpl.rcParams['mathtext.fontset'] = 'stix'
11 mpl.rcParams['axes.unicode_minus'] = False
12
13 mpl.rcParams['font.size'] = 28
14 mpl.rcParams['axes.labelsize'] = 28
15 mpl.rcParams['axes.titlesize'] = 28
16 mpl.rcParams['xtick.labelsize'] = 24
17 mpl.rcParams['ytick.labelsize'] = 24
18 mpl.rcParams['legend.fontsize'] = 22
19
20 # =====
21 # Global Setting
22 # =====
23 SEED = 42
24 np.random.seed(SEED)
25
26 max_iter = 1000
27 eps = 1e-4
28 converge = 1e-9
29 safety = 1e-10
30
31 # =====
32 # Helper functions
33 # =====
34 def normalize_positive(x):
35     x = np.maximum(x, 0)
36     s = np.sum(x)
37     if s < 1e-15:
38         return np.ones_like(x) / len(x)
39     return x / s
40
41 def very_slight_rotation(img2d, angle_deg=1.5):
42     out = rotate(img2d, angle=angle_deg, reshape=False, order=1, mode='nearest')
43     return np.clip(out, 0, None)
44
45 def very_slight_brightness(img2d, factor=1.04):
46     out = img2d * factor
```

```

47     return np.clip(out, 0, None)
48
49 def very_slight_smoothing(img2d, sigma=0.6):
50     out = gaussian_filter(img2d, sigma=sigma)
51     return np.clip(out, 0, None)
52
53 def very_slight_shift(img2d, shift_xy=(1.0, -1.0)):
54     # shift takes (shift_y, shift_x)
55     out = shift(img2d, shift=shift_xy, order=1, mode='nearest')
56     return np.clip(out, 0, None)
57
58 # =====
59 # Import images
60 # =====
61 img_path = r"C:\Users\lenovo\OneDrive\Desktop\涂一辉 机器学习\DBOT\Python算法\image
    \cheetah_64.png"
62 img = Image.open(img_path).convert("L")
63 b1_img = np.array(img, dtype=np.float64)
64
65 # random noise image
66 b2_img = np.random.rand(64, 64)
67
68 # Original flattened vectors
69 b1 = b1_img.reshape(-1)
70 b2 = b2_img.reshape(-1)
71
72 sum1 = np.sum(b1)
73 sum2 = np.sum(b2)
74
75 N = len(b1) # 4096
76
77 # =====
78 # Cost matrix
79 # =====
80 grid_size = 64
81 t = np.linspace(0, grid_size - 1, grid_size)
82 Y, X = np.meshgrid(t, t)
83 position = np.vstack((X.reshape(-1), Y.reshape(-1))).T # shape: (4096, 2)
84
85 # squared Euclidean cost matrix
86 diff = position[:, None, :] - position[None, :, :]
87 C = np.sum(diff**2, axis=2)
88
89 # normalization
90 C = C.astype(np.float64)
91 C = C / np.max(C)
92
93 K = np.exp(-C / eps)
94
95 # =====

```

```

96 # Main Code: Baseline and DBOT
97 # =====
98 def BaselineBarycenter(b1, b2, lambd):
99     N = len(b1)
100     b1 = b1 / b1.sum()
101     b2 = b2 / b2.sum()
102     u1, v1 = np.ones(N), np.ones(N)
103     u2, v2 = np.ones(N), np.ones(N)
104     a = np.ones(N) / N
105
106     starttime = time.time()
107     for _ in range(max_iter):
108         v1 = b1 / (K.T @ u1 + safety)
109         v2 = b2 / (K.T @ u2 + safety)
110         a = (K @ v1) ** lambd * (K @ v2) ** (1 - lambd)
111         a = a / a.sum()
112         u1 = a / (K @ v1 + safety)
113         u2 = a / (K @ v2 + safety)
114         if np.sum(np.abs(u1 * (K.T @ v1) - a)) < converge:
115             break
116     elapse = time.time() - starttime
117
118     P1 = np.outer(u1, v1) * K
119     Wab1 = np.sum(C * P1)
120     P2 = np.outer(u2, v2) * K
121     Wab2 = np.sum(C * P2)
122     Distance = lambd * Wab1 + (1 - lambd) * Wab2
123
124     a_image = a.reshape(64, 64)
125     return a_image, Distance, elapse
126
127
128 def DBOTBarycenter(b1D, b1U, b2D, b2U, sum1, sum2, lambd):
129     N = len(b1D)
130     b1D, b1U = b1D / sum1, b1U / sum1
131     b2D, b2U = b2D / sum2, b2U / sum2
132     u1, v1, q1 = np.ones(N), np.ones(N), np.ones(N)
133     u2, v2, q2 = np.ones(N), np.ones(N), np.ones(N)
134     a = np.ones(N) / N
135
136     starttime = time.time()
137     for _ in range(max_iter):
138         uK1 = u1 @ K
139         uK2 = u2 @ K
140
141         q1 = np.maximum(b1D / (uK1 * v1 + safety), np.ones_like(v1))
142         v1 = np.minimum(b1U / (uK1 * q1 + safety), np.ones_like(v1))
143
144         q2 = np.maximum(b2D / (uK2 * v2 + safety), np.ones_like(v2))
145         v2 = np.minimum(b2U / (uK2 * q2 + safety), np.ones_like(v2))

```

```

146
147     a = (K @ (v1 * q1)) ** lambd * (K @ (v2 * q2)) ** (1 - lambd)
148     a = a / a.sum()
149
150     u1 = a / (K @ (v1 * q1) + safety)
151     u2 = a / (K @ (v2 * q2) + safety)
152
153     if np.sum(np.abs(u1 * (K.T @ (v1 * q1)) - a)) < converge:
154         break
155
156     elapse = time.time() - starttime
157
158     P1 = np.outer(u1, v1 * q1) * K
159     Wab1 = np.sum(C * P1)
160     P2 = np.outer(u2, v2 * q2) * K
161     Wab2 = np.sum(C * P2)
162     Distance = lambd * Wab1 + (1 - lambd) * Wab2
163
164     a_image = a.reshape(64, 64)
165     return a_image, Distance, elapse
166
167 # =====
168 # CV Transformations
169 # =====
170 # Cheetah:
171 #     T1 = very slight rotation
172 #     T2 = very slight brightness adjustment
173 cheetah_t1 = very_slight_rotation(b1_img, angle_deg=1.5)
174 cheetah_t2 = very_slight_brightness(b1_img, factor=1.04)
175
176 # Noise:
177 #     T1 = very slight smoothing
178 #     T2 = very slight shift
179 noise_t1 = very_slight_smoothing(b2_img, sigma=0.6)
180 noise_t2 = very_slight_shift(b2_img, shift_xy=(1.0, -1.0))
181
182 # Flatten
183 cheetah_t1 = cheetah_t1.reshape(-1)
184 cheetah_t2 = cheetah_t2.reshape(-1)
185 noise_t1 = noise_t1.reshape(-1)
186 noise_t2 = noise_t2.reshape(-1)
187
188 # Optional normalization of transformed candidates before min/max
189 cheetah_t1 = normalize_positive(cheetah_t1)
190 cheetah_t2 = normalize_positive(cheetah_t2)
191 noise_t1 = normalize_positive(noise_t1)
192 noise_t2 = normalize_positive(noise_t2)
193
194 # Lower / Upper bounds by elementwise min/max
195 b1d = np.minimum(cheetah_t1, cheetah_t2)

```

```

196 b1u = np.maximum(cheetah_t1, cheetah_t2)
197 b2d = np.minimum(noise_t1, noise_t2)
198 b2u = np.maximum(noise_t1, noise_t2)
199
200 # sums for DBOT normalization
201 sum1_bound = np.sum(b1_img) # keep original style if you want
202 sum2_bound = np.sum(b2_img)
203
204 # 更稳妥一点，也可以这样：
205 sum1_bound = 1.0
206 sum2_bound = 1.0
207
208 # =====
209 # Main Experiment
210 # =====
211 lamdb_list = [0.0, 0.2, 0.4, 0.6, 0.8, 1.0]
212
213 title_text = "实验：从噪声到豹子的平滑过渡"
214 print(title_text.center(130))
215 print("=" * 130)
216
217 baseline_results = []
218 dbot_results = []
219 baseline_times = []
220 dbot_times = []
221 baseline_distances = []
222 dbot_distances = []
223
224 for lamdb in lamdb_list:
225     a_base, d_base, t_base = BaselineBarycenter(b1, b2, lamdb)
226     a_dbot, d_dbot, t_dbot = DBOTBarycenter(b1d, b1u, b2d, b2u, sum1_bound,
227                                             sum2_bound, lamdb)
228
229     baseline_results.append(a_base)
230     dbot_results.append(a_dbot)
231     baseline_times.append(t_base)
232     dbot_times.append(t_dbot)
233     baseline_distances.append(d_base)
234     dbot_distances.append(d_dbot)
235
236     print(
237         f"权重 : {lamdb:.1f} | Baseline 时间: {t_base:.6f}s | DBOT 时间: {t_dbot:.6f}s | "
238         f"Baseline Distance: {d_base:.6f} | DBOT Distance: {d_dbot:.6f}"
239     )
240
241 print("=" * 130)
242 # =====
243 # Visualization 0:

```

```

244 # Upper Bound and Lower Bound under CV Transformation
245 # =====
246 fig0, axes0 = plt.subplots(2, 2, figsize=(10, 10))
247 fig0.suptitle("Upper Bound and Lower Bound under CV Transformation", y=0.98,
248               fontsize=26)
249
250 # Row 1: Cheetah
251 axes0[0, 0].imshow(b1d.reshape(64, 64), cmap='gray')
252 axes0[0, 0].set_title("Lower Bound", pad=10, fontsize=22)
253 axes0[0, 0].axis("off")
254
255 axes0[0, 1].imshow(b1u.reshape(64, 64), cmap='gray')
256 axes0[0, 1].set_title("Upper Bound", pad=10, fontsize=22)
257 axes0[0, 1].axis("off")
258
259 # Row 2: Noise
260 axes0[1, 0].imshow(b2d.reshape(64, 64), cmap='gray')
261 axes0[1, 0].set_title("Lower Bound", pad=10, fontsize=22)
262 axes0[1, 0].axis("off")
263
264 axes0[1, 1].imshow(b2u.reshape(64, 64), cmap='gray')
265 axes0[1, 1].set_title("Upper Bound", pad=10, fontsize=22)
266 axes0[1, 1].axis("off")
267
268 fig0.text(0.04, 0.72, "Cheetah", rotation=90, va='center', ha='center', fontsize
269           =22)
270 fig0.text(0.04, 0.28, "Noise", rotation=90, va='center', ha='center', fontsize=22)
271
272 plt.tight_layout(rect=[0.06, 0.03, 1, 0.94])
273 plt.show()
274
275 # =====
276 # Visualization 1: images
277 # =====
278 fig, axes = plt.subplots(2, len(lambd_list), figsize=(24, 8))
279 fig.suptitle("Smooth Transformation under CV-based Bounds", y=0.98, fontsize=30)
280
281 for j, lambd in enumerate(lambd_list):
282     ax1 = axes[0, j]
283     ax1.imshow(baseline_results[j], cmap='gray')
284     ax1.set_title(rf"$\lambda={lambd:.1f}$", pad=10)
285     ax1.axis("off")
286
287     ax2 = axes[1, j]
288     ax2.imshow(dbot_results[j], cmap='gray')
289     ax2.set_title(rf"$\lambda={lambd:.1f}$", pad=10)
290     ax2.axis("off")
291
292 fig.text(0.04, 0.68, "Vanilla OT", rotation=90,
293         va='center', ha='center', fontsize=24)

```

```
292 fig.text(0.04, 0.27, "DBOT", rotation=90,
293         va='center', ha='center', fontsize=24)
294
295 plt.tight_layout(rect=[0.06, 0.03, 1, 0.94])
296 plt.show()
297
298 # =====
299 # Visualization 2: Distance curve
300 # =====
301 x = np.array(lamdb_list, dtype=np.float64)
302 y_base = np.array(baseline_distances, dtype=np.float64)
303 y_dbot = np.array(dbot_distances, dtype=np.float64)
304
305 x_smooth = np.linspace(x.min(), x.max(), 300)
306
307 if len(x) >= 4:
308     cs_base = CubicSpline(x, y_base)
309     cs_dbot = CubicSpline(x, y_dbot)
310     y_base_smooth = cs_base(x_smooth)
311     y_dbot_smooth = cs_dbot(x_smooth)
312 else:
313     y_base_smooth = np.interp(x_smooth, x, y_base)
314     y_dbot_smooth = np.interp(x_smooth, x, y_dbot)
315
316 plt.figure(figsize=(12, 8))
317
318 plt.plot(
319     x_smooth, y_base_smooth,
320     color='olivedrab',
321     linewidth=3.0,
322     label='Baseline cubic spline'
323 )
324 plt.plot(
325     x, y_base,
326     linestyle='None',
327     marker='^',
328     markersize=10,
329     markerfacecolor='none',
330     markeredgecolor='orange',
331     markeredgewidth=2.2,
332     label='Baseline points'
333 )
334
335 plt.plot(
336     x_smooth, y_dbot_smooth,
337     color='pink',
338     linewidth=3.0,
339     label='DBOT cubic spline'
340 )
341 plt.plot(
```

```
342     x, y_dbot,
343     linestyle='None',
344     marker='s',
345     markersize=9,
346     markerfacecolor='none',
347     markeredgecolor='crimson',
348     markeredgewidth=2.2,
349     label='DBOT points'
350 )
351
352 plt.title("Distance Comparison between Baseline and DBOT", pad=18)
353 plt.xlabel(r"$\lambda$")
354 plt.ylabel("Distance")
355 plt.xticks(lambd_list)
356 plt.grid(True, linestyle='--', alpha=0.4)
357 plt.legend(loc='best', frameon=True)
358 plt.tight_layout()
359 plt.show()
```